

LGD 文件文档

概述

LGD 编译器是一个离线静态编译器 + 静态链接器。

可以把LGC 和 LGD 的关系 类比为 java文件和经过 jvm 编译后的 class文件

最初为早期手游平台设计。目的是将可读的类C脚本（`.lgc`）预编译为机器可执行的二进制字节码（`.lgd`），以换取更快的加载速度和更低的 CPU 开销。加密只是顺手做的副产物罢了

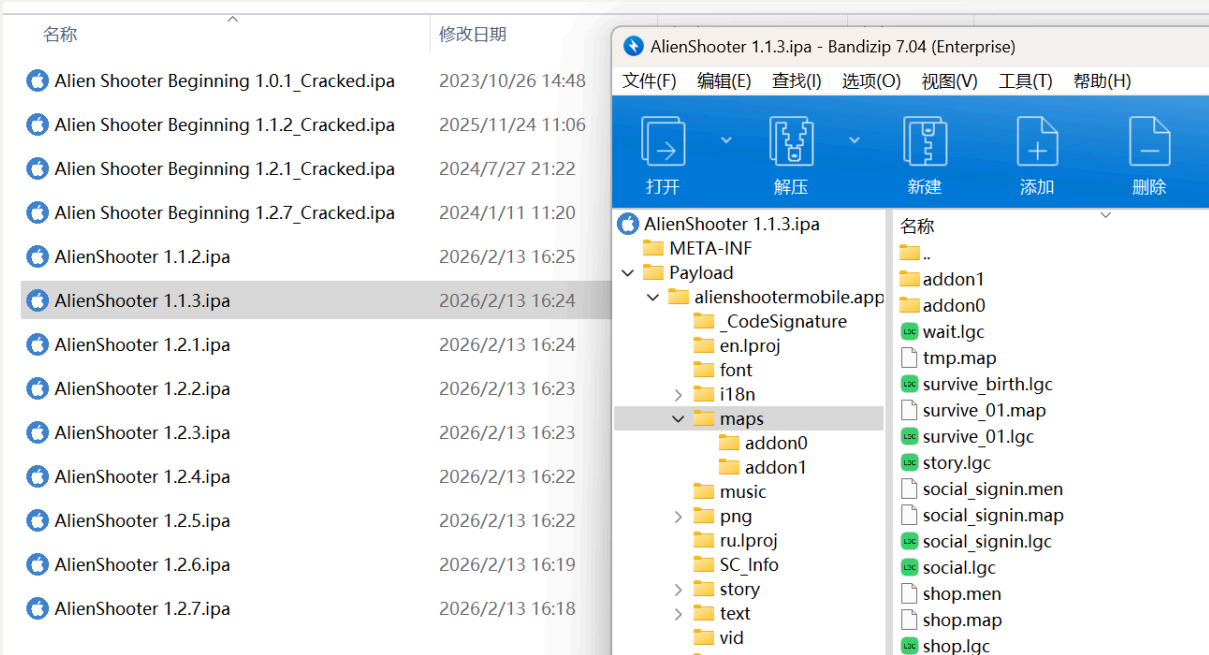
假设关卡脚本 `Level01.lgc` 引用了 `common.lgc` 等 10 个外部脚本。在未编译的情况下，引擎在加载该关卡时需要同时发起 10 次文件 I/O 操作进行读取和解析，效率极低；而在改用 LGD 编译器后，所有依赖均被打包链接成一个独立的 `.lgd` 文件，引擎只需处理单次 I/O 即可完成加载，大幅提升了运行效率。

其编译方式主要包括三个部分：

- 预处理（**Preprocessing**）：递归处理源码中的 `#include` 指令。它会将所有引用的公用脚本（如 `items.lgc`, `common.lgc`）像链接 C 代码一样，“揉”进一个最终的二进制流中。
- 语法校验：具备完整的语法校验功能, 如果有错那会直接报错
- 地图绑定机制：编译过程以 `.map` 文件为索引。采用“一地图、一脚本包”的策略。

Note

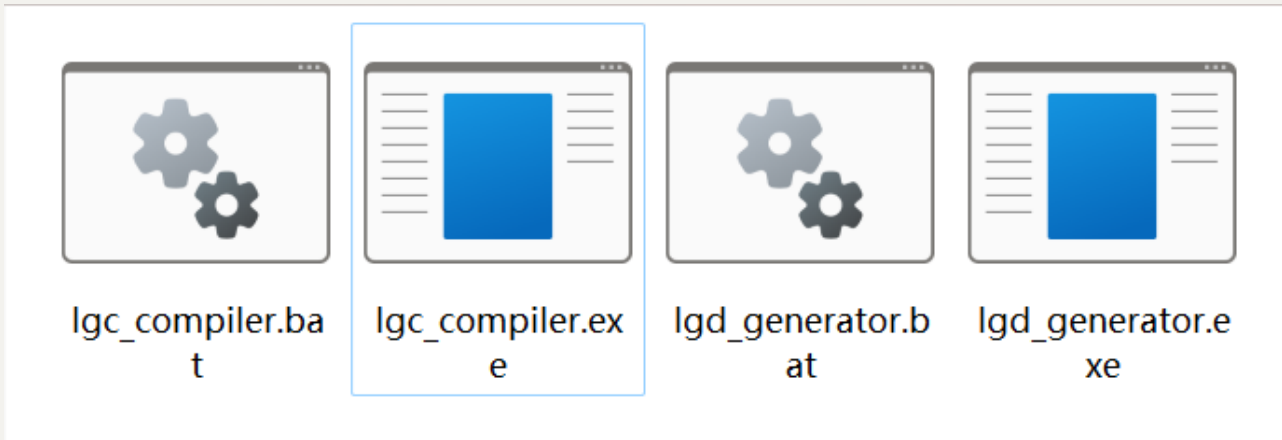
最后一个使用 lgc 的版本是 Alien Shooter The Beginning 1.1.3 IOS VER, 2013年11月22日发布



版本及用法说明

目前我们手上有两个版本的编译器（**sigma**的保密工作做的十分甚至九分地好，时不时送你一点内部工具。当然我怀疑也可能和合作商 **8-floor-games** 有关，和他们合作的版本泄露过不少好东西，所以在大概22-23年之后**sigma**就解除了和他们的合作关系），方便起见就叫他们新版和旧版。本文主要分析都基于新版制作

- 旧版 `lgd_generator.exe` 编译时间 2013-12-14 14:39:05
- 新版 `lgc_compiler.exe` 编译时间 2017-10-31 08:36:40



在运行方式上没有本质上差别，新版多了不少奇怪的功能，不过大概率用不到他们。

新版引擎能兼容旧版旧版生成的LGD

旧版参数

参数	说明
<code>all</code>	编译当前目录及所有子目录下的全部 <code>lgc</code> 文件并生成 <code>lgd</code>
<code>[filename]</code>	仅针对指定的单个 <code>lgc</code> 文件生成 <code>lgd</code>

新版参数

参数	格式	逻辑说明
1. 源码目录	<code>--source-dir=DIR</code>	指定搜索 <code>.lgc</code> 文件的根路径，若不设置则默认为当前目录。
2. 目标目录	<code>--target-dir=DIR</code>	指定生成的 <code>.lgd</code> 文件存放位置，会保留原始目录结构。
3. 宏定义	<code>-D<DEFINE></code>	设置预处理器宏（如 <code>-DPLATFORM_WIN=1</code> ），用于脚本内部的条件编译。
4. 包含路径	<code>-I<PATH></code>	增加 <code>#include</code> 搜索路径，支持多个路径叠加。
5. 冗长输出	<code>-v</code> 或 <code>--verbose</code>	开启详细日志模式，显示编译过程中的符号处理细节。
6. 基础扩展信息	<code>--base-expansion-info</code> <code>PATH</code>	补丁? 指向现有的扩展信息文件，若不存在则会尝试创建。
7. 扩展文件	<code>--expansion-</code> <code>file=PATH</code>	补丁? 指定扩展/补丁文件的路径，触发 <code>Checksum</code> 计算逻辑。
8. 版本号	<code>--version=VERSION</code>	为补丁指定一个版本字符串，存入 <code>EXPANSION_VERSION</code> 。
9. 错误处理	<code>--stop-on-error</code>	遇到第一个编译错误时立即停止，不再处理后续文件。
10. 目标文件	<code>FILES</code>	(位置参数) 指定一个或多个要编译的特定 <code>.lgc</code> 文件。

还有一点，旧版在使用`include`时必须加上`map/`路径

⚠ Warning

关于 `#include` 包含路径的版本差异：

- 旧版编译器在引用依赖时，必须显式加上 `maps/` 相对路径前缀：

```

1 #include "maps\core\export.lgc"
2 #include "maps\common.lgc"
3 #include "maps\common_campaign.lgc"

```

- 新版编译器则严禁携带 `maps/` 路径前缀，必须直接指向子目录或相对路径：

```

1 #include "core\export.lgc"
2 #include "common.lgc"
3 #include "common_campaign.lgc"

```

文件格式

分析之前先随便打开一个文件看看其格式，方便后续分析

可以看到：

第一部分，十分稀疏，基本都是 空文字

tutorial_00.lgd x																	
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0123456789ABCDEF
0:0000	57	07	00	00	82	00	01	00	00	00	00	00	00	00	82	00	W.....
0:0010	01	00	00	00	00	00	00	00	82	00	01	00	00	00	00	00	
0:0020	00	00	82	00	01	00	00	00	00	00	00	00	82	00	01	00	
0:0030	00	00	00	00	00	00	8A	00	01	00	00	00	00	00	00	00	
0:0040	8A	00	01	00	00	00	00	00	00	00	82	00	01	00	00	00	
0:0050	00	00	00	00	82	00	01	00	00	00	00	00	00	00	82	00	
0:0060	01	00	00	00	00	00	00	00	82	00	01	00	00	00	00	00	
0:0070	00	00	8A	00	01	00	00	00	FF	FF	FF	FF	82	00	01	00	
0:0080	00	00	00	00	00	00	82	00	01	00	00	00	00	00	00	00	
0:0090	82	00	01	00	00	00	00	00	00	00	8A	00	01	00	00	00	
0:00A0	00	00	00	00	82	00	01	00	00	00	00	00	00	00	82	00	
0:00B0	01	00	00	00	00	00	00	00	82	00	01	00	00	00	00	00	
0:00C0	00	00	82	00	01	00	00	00	00	00	00	00	82	00	01	00	
0:00D0	00	00	00	00	00	00	82	00	01	00	00	00	00	00	00	00	

第二部分，比较稀疏，可以看到些许乱码字符

tutorial_00.lgd x																																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0:4970	00	01	00	00	00	00	00	00	00	36	03	00	00	64	63	76		6...	dcv													
0:4980	74	7C	58	75	7D	72	74	63	00	02	00	00	00	64	00	00	t Xu}	rtc		d..												
0:4990	00	00	00	00	00	01	00	00	00	00	00	00	00	00	00	00																
0:49A0	00	54	65	72	76	63	72	44	67	65	7E	63	72	00	02	01	.Tervcr	Dge~cr	...													
0:49B0	00	00	41	00	00	00	01	00	00	00	06	00	00	00	00	00	..A.....															
0:49C0	00	00	00	00	00	00	51	7B	76	70	7A	76	79	00	02	01	Q{vpzvy	...														
0:49D0	00	00	42	00	00	00	07	00	00	00	01	00	00	00	00	00	..B.....															
0:49E0	00	00	00	00	00	00	44	67	72	74	7E	76	7B	53	76	7A	Dgrt~v{Svz															
0:49F0	76	70	72	54	78	72	71	71	00	02	00	00	00	43	00	00	vprTxrqq	C..														
0:4A00	00	08	00	00	00	02	00	00	00	00	00	00	00	00	00	00																
0:4A10	00	51	7E	65	64	63	42	79	7E	63	00	02	01	00	00	44	.Q~edcBy~c	D														
0:4A20	00	00	00	0A	00	00	00	02	00	00	00	00	00	00	00	00																
0:4A30	00	00	00	59	72	6F	63	42	79	7E	63	00	02	01	00	00	...YrocBy~c															
0:4A40	45	00	00	00	0C	00	00	00	00	00	00	00	00	00	00	00	E.....															
0:4A50	00	00	00	00	50	72	63	44	67	65	7E	63	72	00	02	00	PrcDge~cr	...														
0:4A60	00	00	46	00	00	00	0C	00	00	00	04	00	00	00	00	00	..F.....															
0:4A70	00	00	00	00	00	00	50	72	63	44	67	65	7E	63	72	44	PrcDge~crD															
0:4A80	74	65	00	02	00	00	00	47	00	00	00	10	00	00	00	03	te.....G.....															
0:4A90	00	00	00	00	00	00	00	00	00	00	00	51	7E	79	73	59	Q~ysY															
0:4AA0	72	76	65	72	64	63	44	67	65	7E	63	72	00	02	01	00	rverdcDge~cr															
0:4AB0	00	48	00	00	00	13	00	00	00	05	00	00	00	00	00	00	.H.....															
0:4AC0	00	00	00	00	00	51	7E	79	73	59	72	76	65	72	64	63	Q~ysYrverdc															
0:4AD0	44	67	65	7E	63	72	24	53	00	02	00	00	00	E5	00	00	Dge~cr\$S.....	..														
0:4AE0	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00	00																

第三部分，FF填充（其实严格意义上说这里有两部分）

tutorial_00.lgd x																																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0:C100	00	03	02	00	00	CC	20	01	00	53	07	00	00	00	00	00		..S.....														
0:C110	00	00	00	00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF																
0:C120	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C130	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C140	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C150	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C160	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C170	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C180	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C190	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C1F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C200	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C210	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																
0:C220	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																

第四部分，稠密，可以看到很多 % 和其他乱七八糟的字节

tutorial_00.lgd x																																		
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F		
0:C900	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF																		
0:C910	FF	FF	FF	FF	FF	FF	FF	FF	FF	D9	24	01	00	01	6F	24																		
0:C920	25	25	24	44	25	25	25	34	09	35	25	25	25	01	6F	24	%%\$D%%4.5%%.o\$																	
0:C930	25	25	24	5F	25	25	25	37	30	3D	3F	25	25	25	01	6F	%%\$_%%70=?%%.o																	
0:C940	24	25	25	24	05	25	25	25	2C	3A	3C	E0	26	25	25	39	%%\$\$.%%,<: .%%9																	
0:C950	2A	25	25	25	01	6F	24	25	25	3A	3C	E2	26	25	25	3A	*%%.o\$%%:< .%%:																	
0:C960	3C	E8	26	25	25	01	6E	24	25	25	01	69	24	25	25	01	< .%%.n\$%%.i\$%%.																	
0:C970	68	24	25	25	01	6B	24	25	25	01	20	25	25	25	01	23	h\$%%.k\$%%.%%. #																	
0:C980	25	25	25	64	03	74	24	25	25	3C	EB	26	25	25	01	74	%%d.t\$%%< .%%.t																	
0:C990	24	25	25	24	5D	25	25	25	00	6A	24	25	25	01	09	25	%%\$]\$%%.j\$%%..%																	
0:C9A0	25	25	01	08	25	25	25	6A	3C	EA	26	25	25	01	74	24	%%..%%j< .%%.t\$																	
0:C9B0	25	25	24	7A	25	25	25	01	75	24	25	25	01	09	25	25	%%z%%.u\$%%..%																	
0:C9C0	25	01	08	25	25	25	6A	3C	F5	26	25	25	01	74	24	25	%.%%j<.&%%.t\$%																	
0:C9D0	25	3A	3C	F4	26	25	25	3A	3C	F2	26	25	25	01	77	24	%:< .%%:< .%%.w\$																	
0:C9E0	25	25	01	76	24	25	25	01	71	24	25	25	01	70	24	25	%%.v\$%%.q\$%%.p\$%																	
0:C9F0	25	01	20	25	25	25	01	23	25	25	25	64	03	7D	24	25	%.%%. #%%d.}\$%																	
0:CA00	25	3C	FD	26	25	25	01	7D	24	25	25	24	5E	25	25	25	%<.&%%.}\$%%\$^%%%																	
0:CA10	00	73	24	25	25	01	09	25	25	25	01	08	25	25	25	6A	.s\$%%..%%.%%j																	

第一部分：字面值表 (Literal Table)

主要负责记录所有变量 -- (全局变量、函数参数、局部变量) 的状态以及它们的初始值。

固定头 4 位为 **index count**

Important

1. 引擎底层实际上仅支持 **int**、**string** 以及它们对应的数组。
2. 虽说你可以在脚本中看到 **boolean** 类型，但他们本质还是 **int** 作者通过宏定义让脚本看上去更接近 C 代码
3. 虽然引擎在底层标志位上预留了对“指针”操作的支持，但官方脚本中从未实际使用过。
4. **static** 和非 **static** 的全局变量在编译后的文件中没有任何区别，这两个关键字的差异仅在作为局部变量时才会体现。

1. 核心概念：Index 与 Global ID 的区别

为了看懂字面值表，必须先理清 **Index** (条目索引) 和 **Global ID** (全局/逻辑 ID) 的区别。这也是整个表中最容易混淆的地方：

- **Index** (条目索引)：代表物理存储的“坑位”。一个基本变量占 1 个 Index；一个长

度为 N 的数组，其每一个元素都会占据 1 个独立的 **Index**。文件开头的 **Entry Count** 指的就是 **Index** 的总数。

- **Global ID (逻辑组 ID)**：代表逻辑上的“归属”。它是按组分配的：
 - 一个普通变量独占 1 个 ID。
 - 一整个数组，无论有多少个元素，都共享同一个 **ID**。
 - 一个函数及其内部的所有参数、局部变量，也全部共享同一个 **ID**。

👉 实验对照：数组的 **Index** 与 **ID** 分配

我们以一个包含 3 个元素的 **int** 数组为例：

```
1 int intArray[3] = { 114, 514, 19198 };
```

在字面值表中，它被拆分成了 3 个 **Index** 条目，但它们共享同一个 **ID**（例如 **ID** 为 9）。请注意观察数组首项与后续项的区别：

```
1 // [GlobalID: 9] 占用了 3 个 Index 条目
2 // Flag | 字符串(空) | Size 或 ID | value
3 8E 00 03 00 00 00 /* Size: 3 */ 72 00 00 00 /*
   val: 114 */
4 8E 00 09 00 00 00 /* ID: 9 */ 02 02 00 00 /*
   val: 514 */
5 8E 00 09 00 00 00 /* ID: 9 */ FE 4A 00 00 /*
   val: 19198 */
```

👉 Tip

数组的首个 **Index** 条目记录的是数组的总长度 (**Size**)；而该数组后续的所有 **Index** 条目，在相同位置记录的则是它们所归属的 **Global ID**。很神奇吧，哈哈 😊。

2. 数据条目结构与 Flag 标志位

通过上述分析，我们可以总结出每个 **Index** 条目的标准内存结构如下：

偏移 (BYTE)	字段名	长度	解析说明
0	Flag	1 byte	变量的状态/类型标志位（详见下方表格）。

偏移 (BYTE)	字段名	长度	解析说明
1 ~ n+1	String Content	n + 1 bytes	变量名或字符串内容，以 00 结尾。如果无字符串则仅为一个 00。(注：内容经过 XOR 17 加密)
n+2 ~ n+5	ID / Size	4 bytes	核心区分位：如果是独立变量或数组首项，这里记录的是 Size ；如果是数组的后续元素，这里记录的是 ID 。如果是方法/EXTERN则以变量的方式记录每个参数和局部变量
n+6 ~ n+9	Value	4 bytes	有符号整数的初始值。如果是字符串类型，此处通常补零为 00 00 00 00。

Flag 标志位 (Byte 0) 构成逻辑：

Flag 的值实际上是由不同权重的属性按位或（OR）组合而成的：

权值 (HEX)	功能分类	含义说明
0x80	基准位	Literal Entry (第一部分字面值的必须标识)
0x01	类型位	属于 String (字符串)
0x02	类型位	属于 Int (整数)
0x04	结构位	属于 Array (数组 [])
0x08	状态位	Initialized (已初始化并赋值)
0x20	指针位	Pointer (指针 *, 官方脚本未使用)
0x40	(旧版)状态位	Uninitialized (未初始化)

根据上述权值组合，反编译中常见的 Flag 常量如下：

- 基础未赋值： 81 (未赋值 string), 82 (未赋值 int), 85 (未赋值 string[]), 86 (未赋值 int[])
- 基础已赋值： 89 (已赋值 string), 8A (已赋值 int), 8D (已赋值 string[]), 8E (已赋值 int[])
- 罕见指针类： A1 (string), A2 (int), A5 (string[]), A6 (int[])

旧版 Flag

- 基础未赋值： C1 (未赋值 string), C2 (未赋值 int), C5 (未赋值 string[]), C6 (未赋值 int[])
- 基础已赋值： 89 (已赋值 string), 8A (已赋值 int), 8D (已赋值 string[]), 8E (已赋值 int[])

int[])

- 罕见指针类：E1 (未赋值 string), E2 (未赋值 int), E5 (未赋值 string[]), E6 (未赋值 int[])

3. 基础类型字节码解析

为了更直观地理解字面值表的结构，这一节将对各种类型的十六进制进行分析。数组已经在前文中讲过了，此处略过。

案例一：基础的 Int 变量

我们定义一个未赋值的 `unusedValue` 和一个赋值为 `166` 的 `gval`：

```
1 int unusedValue;  
2 int gval = 166;
```

对应的字面值表十六进制数据如下：

```
1 // [GlobalID: 0] 变量: int unusedValue;  
2 // Flag | Str | Size(4 bytes) | value(4 bytes)  
3 82 00 01 00 00 00 /* Size: 1 */ 00 00 00 00 /* val:  
  0 */  
4  
5 // [GlobalID: 1] 变量: int gval = 166;  
6 // Flag | Str | Size(4 bytes) | value(4 bytes)  
7 8A 00 01 00 00 00 /* Size: 1 */ A6 00 00 00 /* val:  
  166 (0xA6) */
```

解析：

- **Flag:** `82` 代表未赋值的 int；`8A` 代表已赋值的 int。
- **Str:** 因为是整数变量，这里没有字符串内容，直接用 `00` 占位并结束。
- **Size:** 这是一个单体变量，所以 Size 固定为 1 (`01 00 00 00`)。
- **Value:** `unusedValue` 默认填 0；`gval` 的 166 转换为十六进制即为 `A6 00 00 00`。

案例二：基础的 **String** 变量

字符串的处理方式与整数类似，但它的“值”是直接写入 **str** 字段的（且经过了简单的异或加密），而尾部的 **value** 字段会被强制置零。

```
1 string unusedTxt;  
2 string txt = "sample text";
```

对应的字面值表十六进制数据如下：

```
1 // [GlobalID: 4] 变量: string unusedTxt;  
2 // Flag | Str | Size(4 bytes) | value(4 bytes)  
3 81 00 01 00 00 00 /* Size: 1 */ 00 00 00 00 /* val:  
  0 */  
4  
5 // [GlobalID: 5] 变量: string txt = "sample text";  
6 // Flag | Str (加密后的字符串) | Size(4  
  bytes) | value(4 bytes)  
7 89 64 76 7A 67 7B 72 37 63 72 6F 63 00 /* "sample text" */  
  01 00 00 00 /* Size: 1 */ 00 00 00 00 /* val: 0 */
```

解析：

- **Flag:** 81 代表未赋值 string; 89 代表已赋值 string。
- **Str:** 未赋值时仅为 00。赋值为 "sample text" 时，引擎将其经过 XOR 17 加密后存入（十六进制 64 76...），并在末尾补 00 作为字符串终止符。
- **Value:** 由于数据已经存在了前面的 **str** 块中，后方的 Value 字段统一填充为 00 00 00 00。

案例三：一般方法

```
1 ScriptEvent4( int unit, int var1 = 114, string var2 = "text" ) {  
2     int val = 100;  
3 }
```

```

1 // [GlobalID: 0] -----
2 // Function: ScriptEvent4( int para1, int para2, string para3 )
3 //      (Locals inside this block...)
4 // [Arg 1]
5      82 00                                01 00 00 00 /* Para 1 */      00
6      00 00 00 /* val: 0 */
7 // [Arg 2]
8      8A 00                                01 00 00 00 /* Para 2 */      72
9      00 00 00 /* val: 114 */
10 // [Arg 3]
11      89 63 72 6F 63 00 /* "text" */      01 00 00 00 /* Para 3
12      */      00 00 00 00 /* val: 0 */
13      82 00                                01 00 00 00 /* Para 4 */      00
14      00 00 00 /* val: 0 */

```

正如前文所言，一个方法共享一个 **Global ID** 并且其中记录了其所有参数和全部局部变量

案例四：EXTERN 方法

extern 关键字用于声明引擎底层提供的内置函数。在字面值表中，**extern** 函数的表现与普通函数非常相似：它们同样会被分配一个独立的 **Global ID**。并且记录其参数

```

1 extern MessageText(string text,int x,int y,int fontNvid = -1)
2 114;

```

```

1 // Extern: MessageText (Extern ID: 114)
2      81 00                                01 00 00 00 /* Size: 1 */      00
3      00 00 00 /* string text */
4      82 00                                01 00 00 00 /* Size: 1 */      00
5      00 00 00 /* int x */
6      82 00                                01 00 00 00 /* Size: 1 */      00
7      00 00 00 /* int y */
8      8A 00                                01 00 00 00 /* Size: 1 */      FF
9      FF FF FF /* int fontNvid = -1 */

```

4. 编译器的一些特性

特性一：常量的彻底抹除

表里找不到宏定义或常量？因为编译器在预处理阶段直接把它们“丢弃”并替换成了字面数值。

这倒是挺正常的，不少编译器都有这个特性。

```
1 // 源码
2 #define USED_CONST 150
3 int constVal = USED_CONST;
```

```
1 // 编译后的字面值表：直接当作 int constVal = 150; 处理
2 // Flag | Str | Size | Value
3 8A 00 01 00 00 00 /* Size: 1 */ 96 00 00 00 /* Val:
150 */
```

特性二：局部变量的 **Static** 差异化处理

函数内部的局部变量对 **static** 的处理逻辑截然不同，尤其是数组，十分甚至九分地抽象。

如果是 **static** 局部变量：它会在字面值表中直接保留初始化的值。

```
1 static int stval = 100;
2 static int starr[] = {0x2222, 0x3333, 0x4444, 0x5555};
```

```
1 // 正常保留了数值 (64 00 00 00 即 100, 22 22 00 00 即 0x2222...)
2 8A 00 01 00 00 00 /* Para 4 */ 64 00 00 00 /* val: 100 */
3 8E 00 04 00 00 00 /* Para 5 */ 22 22 00 00 /* val: 8738
(0x2222) */
4 // ...后续数组元素正常带值
```

如果是非 **static** 局部变量：编译器会将其强行降级为未初始化变量，其初始值在字面值表中全部填 **0**。实际的赋值操作被编译器推迟到了后期的 **Bytecode** (字节码) 执行阶段。

```
1 int val = 100;
2 int arr[] = {0x2222, 0x3333, 0x4444, 0x5555};

1 // Flag 变成了 82(未赋值int) 和 86(未赋值int数组), 值全部是 00 00 00
  00!
2      82 00 01 00 00 00 /* Para 9 */ 00 00 00 00 /* val: 0 */
3      86 00 04 00 00 00 /* Para 10 */ 00 00 00 00 /* val: 0 */
4      86 00 11 00 00 00 /* Para 11 */ 00 00 00 00 /* val: 0 */
5      // ...全都是 0
```

第二部分：符号表 (Symbol Table) 结构解析

紧接着字面值表之后的是符号表。它的核心作用是将我们在源码中定义的“变量名”和“函数名”与第一部分分配的 `Global ID` 绑定起来，并记录它们的元数据（如入口偏移量、参数数量等）。

 Tip

 引擎冷知识：局部变量去哪了？

在符号表中，你会发现只有全局变量和函数名的存在。所有的函数参数名、局部变量名全部被编译器丢弃了！在引擎底层的视角里，局部变量只是一堆没有名字的堆栈偏移量和在字面值表中占用的 `Index`。

1. 符号表整体结构

符号表以一个 4 byte 的总数（Entry Count）开头，指示表内共有多少个 Global ID 条目。

紧接着是顺序排列的条目数据。每个条目由 动态长度的字符串 + 固定 24 byte 的属性块 组成：

- 符号名 (String): 经过 XOR 17 加密的字符串，以 00 结尾。
- 属性块 (24 bytes): 固定长度的二进制结构，具体定义如下表。

内部偏移 (BYTE)	长度	字段名	详细含义与处理逻辑
+0	2 bytes	Type	符号类型与状态标志。区分它是变量、普通函数还是 Extern 函数（详见下文状态表）。

内部偏移 (BYTE)	长度	字段名	详细含义与处理逻辑
+2	2 bytes	Meta / Garbage	未初始化栈残留垃圾
+4	4 bytes	Extern ID / Offset / Padding	核心多路复用字段（内置系统 ID / 相对代码段偏移）。 ● Extern 函数 (Type=02): 记录其对应的引擎底层系统函数 ID。 ● 普通函数 (Type=03): 记录该函数在第四部分（Bytecode 代码段）内部对应的字节码相对偏移量 (Relative Offset)。 (公式: 文件的绝对物理偏移 = 代码段在文件中的起始物理偏移 + 相对字节码偏移 = 实际反编译代码中的入口物理地址)。 ● 全局变量 (Type=01): 无实质作用, 固定填充为 00 00 00 00。
+8	4 bytes	P1_Idx	字面值表索引 (Part 1 Index)。指向该符号在第一部分（字面值表）中对应初始值的起始位置。
+12	4 bytes	Size	大小 / 元素数。 ● 变量: 数组的长度（如 <code>int a</code> 为 1, <code>int a[3]</code> 为 3）。 ● 函数: 参数列表的总长度（注意：不包含局部变量）。
+16	4 bytes	File_ID	源文件 ID。 ● 分配机制: 严格从 0 开始递增。0 代表主文件, 1, 2... 代表预处理展开时, 按物理首次被打开的先后顺序被赋予每个被包含 (<code>#include</code>) 的文件。 ● 断层/跳跃解释: 符号表里常看到 0, 12, 13... 等不连续的 File_ID, 是因为中间跳过的文件（如 1~11 号）是未定义全局变量的文件, 它们在符号表里没有任何条目输出, 从而造成了表面的断层现象。 ● 注意: 仅全局变量的 File_ID 不为 0 其他项目均为 0。

内部偏移 (BYTE)	长度	字段名	详细含义与处理逻辑
+20	4 bytes	Run_ID	编译阶段局部槽位索引 (Run_ID)。在反编译和运行时无实质作用。 ● 文件局部性：它是全局变量在所属源文件（File_ID）内部独立的局部槽位索引。由于不同源文件各自递增分配，跨文件时 Run_ID 必然发生重复。 ● 非变量置零：只有全局变量符号此值不为零，所有的普通函数、外部函数 (Extern) 等此值统一重复为 0。 ● 逆向结论：引擎在运行字节码时完全通过全局唯一的 P1_Idx 进行内存寻址，Run_ID 仅为编译期残留产物，逆向分析时可以完全忽略。

2. Type 标志位与“返回值追踪”机制

符号表 +0 偏移处的 **Type** 字段不仅仅是用来区分变量和函数的，它还隐藏了一个极其精妙的机制：编译器会静态分析函数的返回值是否在上下文中被实际使用，并将其记录在 **Type** 中。

Type 标志位枚举大全：

- **01 00** : 全局变量 (Variable)
- **02 00** : 外部函数 (Extern)，且返回值未被使用
- **02 01** : 外部函数 (Extern)，且返回值被使用
- **03 00** : 普通函数 (Function)，无返回值 (Void)
- **03 01** : 普通函数 (Function)，有返回值，且被 **await** 关键字使用
- **03 02** : 普通函数 (Function)，有返回值，但未被使用
- **03 03** : 普通函数 (Function)，有返回值，且被使用

👉 实验对照：**Extern** 返回值追踪

以 **Createsprite** 这个 **extern** 函数为例，调用方式的不同会直接改变其在符号表中的 **Type** 签名：


```

1 // 场景 A: 仅仅调用, 不管返回值
2 CreateSprite(1, 10, 20, 0);
3 // 对应 Type -> 02 00
4
5 // 场景 B: 将返回值赋给变量
6 int s = CreateSprite(1, 10, 20, 0);
7 // 对应 Type -> 02 01

```

3. Size 的真正含义：参数与局部的分界线

对于变量，**size** 很好理解（就是数组长度）；但对于函数，**Size** 字段仅仅统计函数参数的数量，完全忽略内部的局部变量。

👉 实验对照：函数 **Size** 统计逻辑

我们依旧定义如下代码：

```

1 ScriptEvent4( int unit, int var1 = 114, string var2 = "text" ) {
2     static int stval = 100;
3     static int starr[] = {0x2222, 0x3333, 0x4444, 0x5555};
4     int val = 100;
5     int arr[] = {0x2222, 0x3333, 0x4444, 0x5555};
6 }
7
8 int afterFun[] = {2, 3, 4};

```

分析编译后的符号表条目：

1. 函数 **ScriptEvent4**：

```

1 // 加密名: ScriptEvent4
2 44 74 65 7E 67 63 52 61 72 79 63 23 00
3 03 00 // Type: Function (Void 无返回值)
4 C9 00 // Meta
5 00 00 00 00 // Padding: 0
6 14 00 00 00 // P1_Index: 20
7 03 00 00 00 // Size: 3 (核心验证点: 仅统计了 unit, var1, var2 三个
   参数)
8 00 00 00 00 // FileID: 0
9 00 00 00 00 // RunID: 0

```

2. 全局变量 afterFun:

```

1 // 加密名: afterFun
2 76 71 63 72 65 51 62 79 00
3 01 00 // Type: Variable
4 CA 00 // Meta: 垃圾残留数据 (Garbage)
5 00 00 00 00 // Padding: 0
6 21 00 00 00 // P1_Index: 33
7 03 00 00 00 // Size: 3 (数组长度为 3)
8 00 00 00 00 // FileID: 0
9 1A 00 00 00 // RunID: 26

```

正如所见，尽管 `ScriptEvent4` 内部塞满了 `stval`、`starr`、`val` 等局部变量，占据了大量的 `P1_Index` 空间，但符号表中的 `Size` 依然显示为 `03`。这就证实了符号表对函数的管理是处于“外部调用签名”层面，并不关心其内部的局部状态。不过我们仍可以通过上下两组的 `index` 差 并综合 `P1` 来推算一个函数内局部变量的情况。

第三部分：特殊值表 (Action & ScriptEvent Table)

一块默认全是 `FF` 填充的区域，给 `ActionX` 和 `ScriptEventX` 使用，可能是用于快速定位以及和地图交互

这个特殊值表的尺寸和结构是绝对固定的，总大小为 **2048 Bytes**。计算公式为：**4 bytes** (单个坑位大小) × **256** (支持的编号数量 0~255) × **2** (变量与函数两组)。

如果在 `.lgc` 脚本中定义了这些带有特殊编号的变量或函数，编译器就会在编译时，将这块区域对应编号的 `FF FF FF FF` “挖空”，并填上它们的引用地址。

样例代码

```
1  int arr[] = {1,2,3}; //id = 0 index = 0-3
2  int a = 114; //id = 1 index = 4
3
4  int Action2 = 100; //id = 2 index = 5
5  int Action5 = 200; //id = 3 index = 6
6
7  int arr2[] = {1,2,3}; //id = 4
8
9  ScriptEvent1( int val1, int val2, int val3 ){
10     //id = 5
11 }
12
13 ScriptEvent3( int val1, int val2, int val3 ){
14     //id = 6
15 }
```

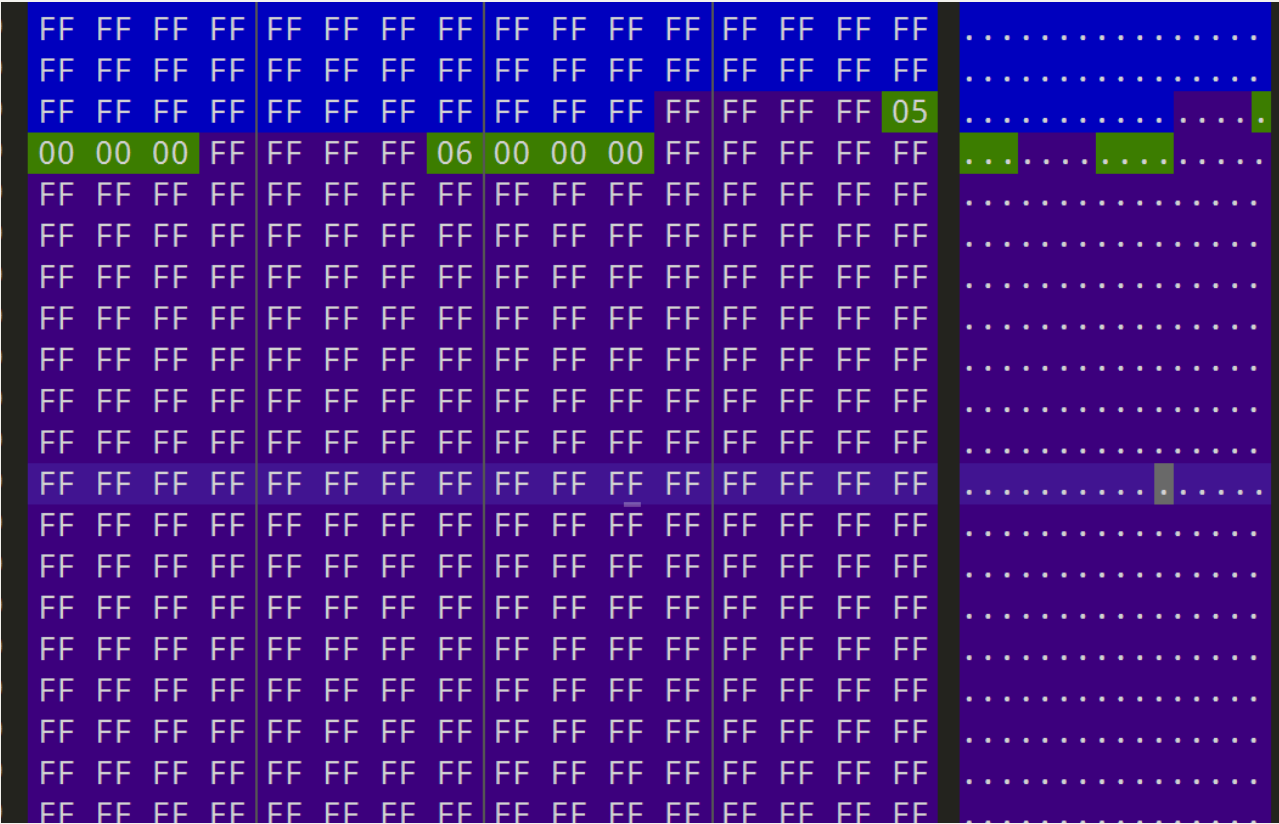
规则 A: 特殊变量 `int ActionX`

- 类型限制：必须是 `int` 类型。
- 填入内容：记录该变量在字面值表 (Part 1) 中的 **P1_Index** (条目索引)。
- 举例：如代码所示，定义了 Action2/Action5 那么就把对应的FF填充挖空并填上 index 数值

0150	00	00	00	00	00	00	44	74	65	7E	67	63	52	61	72	79	Dte~gcRar
0160	63	24	00	03	00	68	00	01	00	00	00	0C	00	00	00	03	c\$...h.....
0170	00	00	00	00	00	00	00	00	00	00	00	FF	FF	FF	FF	FF	
0180	FF	FF	FF	04	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	05	
0190	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01A0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01B0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01C0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01D0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01E0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
01F0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0200	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0210	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0220	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0230	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0240	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0250	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0260	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
0270	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	

规则 B：特殊函数 `ScriptEventX`

- 参数强制限制：必须包含且仅包含 **3** 个参数，例如 `ScriptEvent5(int var1, int var2, int var3)`。如果参数数量不对，编译就不给过
- 填入内容：记录该函数在字面值表中的 **Global ID (全局 ID)**。
- 举例：如代码所示，定义了 `ScriptEvent1/ScriptEvent3` 那么就把对应的FF填充挖空并填上Global ID数值



第四部分：Bytecode 代码段与指令集 (Opcode Table)

代码段的开头由一个 4 byte 的整数 构成，表示后续字节码的总长度

这开头的 4 个字节是明文不加密的，但从第 5 个字节开始的所有字节码数据，全部经过了 XOR 0x25 加密。

指令集如下

1. Value Types & Constants (值类型与常量)

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS (SIZE & TYPE)
0x01	PUSH_INT		将整型常量值压入栈中。	4 Bytes (32位整型常量值)
0x02	PUSH_STR		将字符串常量值压入栈中。	不定长 (以 0x00 结尾的字符串)

2. Unary Operators (一元操作符)

这类操作符直接对当前栈顶的数据进行操作，因此指令本身不带操作数（0 Bytes）。

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS
0x03	NEG	-	算术取负。	0 Bytes (依赖栈)
0x04	BIT_NOT	~	按位取反。	0 Bytes (依赖栈)
0x05	LOG_NOT	!	逻辑非。	0 Bytes (依赖栈)

3. Arithmetic Operators (算术操作符)

二元操作符，从栈中弹出两个值计算后将结果压栈，指令本身无操作数。

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS
0x06	DIV	/	执行除法运算。	0 Bytes
0x07	MOD	%	执行取模运算。	0 Bytes
0x08	ADD	+	执行加法运算。	0 Bytes
0x09	SUB	-	执行减法运算。	0 Bytes
0x0A	BIT_XOR	^	执行按位异或运算。	0 Bytes
0x0B	BIT_OR		执行按位或运算。	0 Bytes
0x0C	BIT_AND	&	执行按位与运算。	0 Bytes
0x13	MUL	*	执行乘法运算。	0 Bytes
0x16	SHR	>>	执行按位右移运算。	0 Bytes
0x17	SHL	<<	执行按位左移运算。	0 Bytes

4. Comparison Operators (比较操作符)

比较栈顶的两个值，将布尔结果压栈，无内联操作数。

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS
0x0D	EQ	==	比较是否等于。	0 Bytes
0x0F	GT	>	比较是否大于。	0 Bytes
0x10	LT	<	比较是否小于。	0 Bytes
0x11	GE	>=	比较是否大于等于。	0 Bytes
0x12	LE	<=	比较是否小于等于。	0 Bytes
0x14	NE	!=	比较是否不等于。	0 Bytes

5. Logical Operators (逻辑操作符)

逻辑计算最终出栈合并结果，指令无内联操作数。

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS
0x0E	LOG_OR		计算逻辑或 (<code>c = a b;</code>)，并将结果压栈。	0 Bytes
0x15	LOG_AND	&&	计算逻辑与 (<code>c = a && b;</code>)，并将结果压栈。	0 Bytes

6. Flow Control & Structure (流控制与结构)

包含跳转、函数调用、等待等机制，大部分带有 4 字节的地址/偏移量或 ID。

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS (SIZE & TYPE)
0x18	JMP_FALSE		如果栈顶值为假，则跳转到指定地址（常用于 if/while）。	4 Bytes (相对偏移量 / 目标地址)
0x19	LINE_NUM	;	标记语句结束，并记录当前源代码的行号。	4 Bytes (代码行号)
0x1A	ARG_COMMIT	()	向非外部 (extern) 函数提交参数。不单独出现，总是与 <code>CALL_FUNC</code> 配合。	0 Bytes

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS (SIZE & TYPE)
0x1B	AWAIT	await	挂起脚本执行（异步等待）。虽有分号但不记录行号，操作数代表括号内语句的偏移量。	4 Bytes (偏移量 offset)
0x1C	JMP		无条件跳转到目标地址（常用于 else/goto/break）。	4 Bytes (相对偏移量 / 目标地址)
0x1D	IFF	iff	执行特殊的条件检查。格式特殊，后跟跳转的目标地址。	8 Bytes (两个 4 字节的目标地址)
0x1E	CALL_FUNC	()	执行一个函数。	4 Bytes (函数 ID 或地址)
0x1F	RET	return	终止当前函数，返回控制权或返回值。	0 Bytes
0x2C	AND_JMP	&&	短路逻辑：如果左操作数为假，立即跳转至失败标签，提前跳过右边操作数的计算。	4 Bytes (相对偏移量)
0x2D	OR_JMP		短路逻辑：如果左操作数为真，立即跳转至成功标签，提前跳过右边操作数的计算。	4 Bytes (相对偏移量)

7. Variables, Memory & Assignment (变量、内存与赋值)

除了对数组访问使用栈中索引外，变量的操作大都需要一个 4 字节的变量标识符 (*Variable ID / Address*)。

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS (SIZE & TYPE)
0x20	POST_INC	i++	变量后置自增。	4 Bytes (变量 ID)
0x21	POST_DEC	i--	变量后置自减。	4 Bytes (变量 ID)
0x22	PRE_INC	++i	变量前置自增。	4 Bytes (变量 ID)
0x23	PRE_DEC	--i	变量前置自减。	4 Bytes (变量 ID)
0x24	PUSH_VAR		将参数或变量的值压入栈中。	4 Bytes (变量 ID)
0x25	ADDR_OF	&	获取变量的内存地址。	4 Bytes (变量 ID)

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS (SIZE & TYPE)
0x26	ASSIGN	=	将栈顶的值赋给指定变量。	4 Bytes (变量 ID)
0x27	OP_ASSIGN		复合赋值标记（由紧跟的下一个字节决定具体操作，如 +=, -= 等）。	5 Bytes (4字节变量 ID + 1字节操作码)
0x28	ARRAY_IDX	[]	使用栈中弹出的索引值访问数组元素。	0 Bytes (依赖栈中弹出的索引)

8. Type Casting (类型转换)

OPCODE	MNEMONIC	SYMBOL	DESCRIPTION (描述)	OPERANDS
0x29	CAST_STR	(string)	将当前栈顶的值转换为字符串类型。	0 Bytes
0x2A	CAST_INT	(int)	将当前栈顶的值转换为整型。	0 Bytes
0x2B	CAST_SPR	(sprite)	[已移除] 将当前栈顶的值转换为精灵(Sprite)对象。	0 Bytes

调用机制与特殊结构

- 当虚拟机需要操作变量时（如 `PUSH_VAR`），它使用的是变量在第一部分的物理索引 `P1_Index`。
- 当虚拟机需要调用函数时（如 `CALL_FUNC`），它使用的是函数在符号表中的 `Global ID`。
- 引擎只能调用在他前面已经定义过的函数,不能调用在其之后定义的.如果发生了此种调用则语法检测会报 `Undeclared Identifier` 错误
- 每一个函数最后都会有一个`RET`结尾,他们没有其他任何意义,引擎占位符,用于标志一个函数的结束
- 反编译时将以函数为最小单位进行按区块反编译。不可能出现跨越方法的`JMP`, 因为所有调取其他方法都有现成的操作码比如`CALL_FUNC`, `call_ext_xx`

iff的结构

iff 后面跟 4 + 4 操作数 第一个表示其为false后需要跳转的地址,第二个则是其判断条件开始时的地址 (可以理解为小括号里的内容)

```
1  loc_0C1A:
2      0C1A:  24 00 00 00 00      PUSH_VAR    g_int
3      0C1F:  01 33 33 33 33      PUSH_INT    0x33333333
4      0C24:  0F                                GT
5      0C25:  1D 1D 00 00 00 0E      IFF          loc_0C43, loc_0C1A  ;
      Iff Jump
6          03 00 00
7      0C2E:  24 00 00 00 00      PUSH_VAR    g_int
8      0C33:  26 05 00 00 00      ASSIGN      sub_func_arg0
9      0C38:  1A                                ARG_COMMIT
10     0C39:  1E 04 00 00 00      CALL_FUNC    sub_func
11     0C3E:  19 4E 00 00 00      LINE_NUM     0x4E                ;
      Code Line 78
12  loc_0C43:
```

对应的源码

```
1      iff (g_int > 0x33333333) {
2          sub_func(g_int);
3      }
```

extern函数调用

如果 Opcode 大于 0x40 (十进制 64), 说明这不是一个内置指令, 而是一个 **Extern** 函数调用。指令 0x41 ~ 0xFF (65~255) 会直接映射到对应的 call_ext_XX 方法

💡 Tip

冷知识, 引擎各个版本export函数数量基本是固定的,大体上也基本一样

范围从 65-255

怀疑当初就给他们安排了 1byte的空闲空间

什么?不够了怎么办?其他的全部扔到i_send_command.h中去 🤪

其参数使用也和一般函数以来ARG_COMMIT不一样,直接把所有需要的参数推进去然后调用

```
1 CreateText(int font_vid,int x,int y,int z,string text,int
  behave=0)
2 {
3     int sprite;
4     sprite = CreateSprite(font_vid,x,y,z);
5     Action(sprite,ACT_SET_TEXT,&text);
6     Action(sprite,ACT_SET_BEHAVE,behave);
7     return sprite;
8 }
```

比如这个例子中使用了 CreateSprite 有5个参数,但只提供了3个,所以在调用时会额外推送两个归属于 CreateSprite 的参数 (存在默认值) 进去来填充空间

```
1 ;Function CreateText() Begin
2   C96A:  19 CD 03 00 00      LINE_NUM      0x3CD          ;
   Code Line 973
3   C96F:  24 4B 01 00 00      PUSH_VAR      CreateText_arg0
4   C974:  24 4C 01 00 00      PUSH_VAR      CreateText_arg1
5   C979:  24 4D 01 00 00      PUSH_VAR      CreateText_arg2
6   C97E:  24 4E 01 00 00      PUSH_VAR      CreateText_arg3
7   C983:  24 05 00 00 00      PUSH_VAR      CreateSprite_arg4
8   C988:  24 06 00 00 00      PUSH_VAR      CreateSprite_arg5
9   C98D:  41                  CALL_EXT_65      ;
   CreateSprite
10  C98E:  26 51 01 00 00      ASSIGN        CreateText_local0
11  C993:  19 CE 03 00 00      LINE_NUM      0x3CE          ;
   Code Line 974
12 ; ...
13  C9E1:  1F                  RET              ;
   mark end of function
14 ;Function CreateText() End
15 ; -----
```

局部非 static 变量初始化

还记得一开始说的非 static 变量初始化吗

最抽象的一部分来了

```
1 ScriptEvent4( int unit, int var1 = 114, string var2 = "text" ){
2     static int stval = 100;
3     static int starr[] = {0x2222, 0x3333, 0x4444, 0x5555};
4     int val = 100;
5     int arr[] = {0x2222, 0x3333, 0x4444, 0x5555};
6 }
7
```

非数组还好说，挺正常的push+assign

```
1 0B9D: 01 64 00 00 00 PUSH_INT 0x64
2 0BA2: 26 1C 00 00 00 ASSIGN ScriptEvent4_local5
3 0BA7: 19 16 00 00 00 LINE_NUM 0x16 ;
Code Line 22
4 0BAC: 19 17 00 00 00 LINE_NUM 0x17 ;
Code Line 23
```

数组...不好评价 把其中每一个元素当成单独的int单独拿出来赋值,赋值完了再组装回去

1	0BB1:	01 22 22 00 00	PUSH_INT	0x2222	
2	0BB6:	26 1D 00 00 00	ASSIGN	ScriptEvent4_local6	
3	0BBB:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					
4	0BC0:	01 33 33 00 00	PUSH_INT	0x3333	
5	0BC5:	26 1E 00 00 00	ASSIGN	ScriptEvent4_local7	
6	0BCA:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					
7	0BCF:	01 44 44 00 00	PUSH_INT	0x4444	
8	0BD4:	26 1F 00 00 00	ASSIGN	ScriptEvent4_local8	
9	0BD9:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					
10	0BDE:	01 55 55 00 00	PUSH_INT	0x5555	
11	0BE3:	26 20 00 00 00	ASSIGN	ScriptEvent4_local9	
12	0BE8:	19 17 00 00 00	LINE_NUM	0x17	;
Code Line 23					

栈机制以及衍生 Bug

正如前文所述，整个 LGD 脚本在虚拟机中是基于栈（**Stack**）结构来运行的。

虚拟机栈的工作原理

引擎在执行字节码时，其核心是一个“后进先出”的求值栈：

- 压栈（**PUSH**）：当遇到常量（如 **PUSH_INT**）或变量（如 **PUSH_VAR**）时，虚拟机会把它们值推入栈顶。
- 计算与弹栈（**POP**）：当遇到操作符（如加减乘除、比较、逻辑运算）时，虚拟机会从栈顶弹出所需数量的操作数，计算完毕后再将结果压回栈顶。
 - 例如：计算 **a + b** 时，先 **PUSH a**，再 **PUSH b**，然后遇到 **ADD** 指令，弹出两者，相加后把结果压回栈顶。
- **Extern** 调用：在调用 **Extern** 函数时，虚拟机会连续推入参数，然后一次性由函数体弹出使用。

栈相关的经典 Bug

既然是栈，那肯定有栈相关的bug:

1. 栈下溢 (**Stack Underflow**): 虚拟机尝试执行某个需要操作数的指令时，栈里却没有足够的空间（比如执行 `ADD` 但栈里空无一物）。不多见，一般这种情况编译就不过。
2. 栈上溢 (**Stack Overflow**): 短时间内向栈内推入过多数据，超出了虚拟机栈的深度上限。同样不多见。
3. 栈失衡 (**Stack Out / Stack In**): 隐藏 Bug。编译通过，有时候游戏也没问题(甚至可见于官方脚本)。编译器设计时的检测不严格。

孤立表达式引发的Stack Out

源码

AS-Free 4.5.3 源码

```
1 // 某些特定菜单不需要弹出升级确认对话框
2 isSkipConfirmationLevelUpFor(int a_currentMenu)
3 {
4     if ( (currentMenu == MENU_CHARACTERS_INFO)
5         || (currentMenu == MENU_CHARACTER_SKILLS)
6         || (currentMenu == MENU_SHOP)
7         || (currentMenu == MENU_INVENTORY)
8         || (currentMenu == MENU_MISSION_INFO)
9     )
10     true; // ⚠ 可能是单纯的漏写了return谁知道呢 ⚠
11         // 虽然语法合法不报错，但它没有任何实际意义。
12
13     return false;
14 }
```

汇编代码

```
1 ;Function isSkipConfirmationLevelUpFor() Begin
2 556DF: 24 E7 0A 00 00    PUSH_VAR    currentMenu
3 556E4: 01 73 00 00 00    PUSH_INT    115
4 556E9: 0D                EQ
5 556EA: 2D 10 00 00 00    OR_JMP      loc_556FB    ;
    短路逻辑跳转
6 ;... （省略中间的多次逻辑或运算） ...
7 loc_5572E:
8 5572E: 18 0E 00 00 00    JMP_FALSE   loc_5573D    ;
    若 if 条件不成立，跳过
9 55733: 01 01 00 00 00    PUSH_INT    1            ;
    ⚠ 编译 true; 产生的压栈操作！这个 1 随后再未被使用
10 55738: 19 0F 00 00 00    LINE_NUM    15          ;
    Code Line 15
11 loc_5573D:
12 5573D: 01 00 00 00 00    PUSH_INT    0            ;
    压入 return false 的返回值 0
13 55742: 1F                RET                      ;
    执行返回，弹出栈顶的 0 并返回
14 55743: 19 11 00 00 00    LINE_NUM    17          ;
15 55748: 1F                RET                      ;
    mark end of function
16 ;Function isSkipConfirmationLevelUpFor() End
```

反编译代码

```
1  isSkipConfirmationLevelUpFor(int
   isSkipConfirmationLevelUpFor_arg0)
2  {
3      if ((((((currentMenu == 115) || (currentMenu == 117)) ||
   (currentMenu == 104)) || (currentMenu == 118)) || (currentMenu ==
   100)))) {
4          // WARNING: Orphaned expression -> Stack_Out = 1
5          // --- Line 15 ---
6      }
7      return 0;
8      // --- Line 17 ---
9  }
```

Bug 发生原理

1. 当 `if` 条件不成立时：

- 直接跳到 `loc_5573D`，压入 `0`，执行 `RET` 弹出 `0` 并正常返回。此时栈是平衡的。

2. 当 `if` 条件成立时：

- 执行 `PUSH_INT 1`，将 `1` 压入栈顶。
- 因为没有 `else`，程序顺次往下执行到 `loc_5573D`，再次执行 `PUSH_INT 0`。
- 此时，栈里同时存在了 `1` 和 `0` 两个数据！
- 紧接着执行 `RET`，虚拟机弹出栈顶的 `0` 作为函数的返回值成功返回。
- 但是，先前压入的 `1` 却被遗忘在了栈里，没有被清空。

这个“被遗忘的 1”就是反编译器中警告的 `Orphaned expression -> Stack_Out = 1`（孤立表达式产生的栈残留）。如果该函数在游戏主循环中被反复调用，虚拟机的求值栈终将因为溢出而使游戏彻底崩溃。

悬空操作符引发的StackIn

在官方脚本中，还会遇到另一种与 `#ifdef` 相关的 Bug。

这个遇到次数更少，只在测试 AS-Free 2.0.4 时遇到过一次，遗憾的是那个版本并没有给出源代码，所以只能通过一些模拟测试脚本来推演这个情况。

源码

```
1 test_ifdef_bug_and_end()
2 {
3     int money = 10;
4
5     // 如果 TARGET_PLATFORM_IOS 没有定义
6     if ( money > 100 &&
7 #ifdef TARGET_PLATFORM_IOS
8         CheckIOSPurchase()
9 #endif
10    )
11    {
12        money = 0;
13    }
14 }
```

在正经的 C++ 编译器中，如果 `TARGET_PLATFORM_IOS` 未定义，预处理展开后会变成 `if ( money > 100 && )`，由于缺少右操作数，这在编译期就会被直接拒绝。

然而，简易的 LGC 编译器不仅成功通过了编译，还生成了如下神奇的字节码：

汇编代码

```
1 ;Function test_ifdef_bug_and_end() Begin
2 09DA: 01 0A 00 00 00    PUSH_INT    10
3 09DF: 26 00 00 00 00    ASSIGN
   test_ifdef_bug_and_end_local0
4 09E4: 19 11 00 00 00    LINE_NUM    17
5 09E9: 19 12 00 00 00    LINE_NUM    18
```

```

6  09EE: 24 00 00 00 00      PUSH_VAR
    test_ifdef_bug_and_end_local0 ; 压入 money (10)
7  09F3: 01 64 00 00 00      PUSH_INT      100
    ; 压入 100
8  09F8: 0F                      GT
    ; 计算 money > 100 (结果为假/0)
9  09F9: 2C 05 00 00 00      AND_JMP      loc_09FF      ;
    ⚠ 短路与: 若栈顶为假则跳转至 loc_09FF
10 09FE: 15                      LOG_AND
    ⚠ 逻辑与: 需要弹出两个操作数!
11 loc_09FF:
12 09FF: 18 13 00 00 00      JMP_FALSE   loc_0A13      ;
    若条件为假, 跳过 if 语句块
13 0A04: 01 00 00 00 00      PUSH_INT      0
14 0A09: 26 00 00 00 00      ASSIGN
    test_ifdef_bug_and_end_local0
15 loc_0A13:
16 0A13: 1F                      RET
17 ;Function test_ifdef_bug_and_end() End

```

Bug 发生原理

LGC 编译器对逻辑与 `&&` 采用的是短路求值 (Short-Circuit Evaluation) :

1. `AND_JMP` (操作码 `2C`) : 如果左侧条件为假 (0), 立即短路跳转到 `loc_09FF`, 提前结束 `&&` 后续的求值。
2. `LOG_AND` (操作码 `15`) : 如果左侧为真 (1), 则继续执行后续表达式, 并最终调用 `LOG_AND` 弹出栈顶的两个值做逻辑与。

但因为 `#ifdef` 中间的内容被完全掏空, `&&` 后面已经没有任何可以执行的表达式了。于是发生了极其荒谬的事情:

- 路径 A: 当 `money` 为 `10` 时 (`money > 100` 为假)
 - a. `GT` 结果为 `0` 并压入栈顶。
 - b. `AND_JMP` 判断栈顶为 `0`, 触发短路跳转, 直接飞到了 `loc_09FF` !
 - c. 它成功绕过了底下的 `09FE: 15 LOG_AND` 字节码。
 - d. 接着 `JMP_FALSE` 读到栈顶的 `0`, 跳过 `if` 块。

- e. 结果：因为短路跳转直接绕过了那条缺操作数的 `LOG_AND` 指令，栈结构没有崩溃，程序奇迹般地完全正常运行！这就是所谓的“**Stack In**”侥幸成立状态。
- 路径 B：当 `money` 被设为 `200` 时（`money > 100` 为真）
 - a. `GT` 结果为 `1` 并压入栈顶。
 - b. `AND_JMP` 读到 `1`，不触发跳转，程序被迫顺次往下执行。
 - c. 虚拟机执行到下一条指令 `09FE: 15 LOG_AND`。
 - d. `LOG_AND` 是一个二元操作符，它会尝试从栈中弹出两个值进行逻辑与。
 - e. 但此时栈中只有一个计算出来的 `1`（因为后续的表达式被掏空了，压根没往栈里压入第二个值）！
 - f. 结果：栈里只有一个操作数，`LOG_AND` 强行弹栈，触发 栈下溢（**Stack Underflow**），或者弹出了栈底无关的其它数据，导致虚拟机发生不可预测的崩溃！

同样的道理也适用于逻辑或 `||`（对应短路跳转 `OR_JMP` 和二元操作符 `LOG_OR`）。

悬空二元操作符的通用性与编译器语法校验规则

需要强调的是，这个“悬空操作符”漏洞绝不仅仅存在于 `&&` 和 `||` 逻辑操作符上，它适用于所有的二元操作符（如 `+`, `-`, `*`, `/`, `==`, `>`, `<` 等）。

规律如下

规则 A：放行编译成功——操作符紧邻括号

当悬空的二元操作符紧贴着界符（括号 `(` 或 `)`）时，编译器会直接判定语法合法并放行：

不管怎么说至少编译是能过的，反编译均产生 `stack_in`

- 操作符挨着右括号：`if ( A && )` ✓
- 操作符挨着左括号：`if ( || B )` ✓
- 嵌套括号的悬空：`if ( ( A || B ) && )` ✓

底层逻辑：

编译器的语法树并不完善。它只校验括号的配对和表达式的整体边界，但完全没有校验二元操作符的两侧是否都连结着合法的子表达式/操作数。它只要遇到括号，就默认这里是表达式的边界，直接无视了悬空操作符的缺陷，照常生成对应的二元计算字节码（如 `LOG_AND`, `LOG_OR`, `ADD` 等）。

规则 B：拒绝编译报错——操作符紧邻操作符

如果表达式中出现了两个二元操作符紧紧挨在一起的情况，比如中间连续两个悬挂 `#ifdef` 编译器会直接报错

- 不同操作符挨着：`if ( A && || B )` ❌
- 相同操作符挨着：`if ( A && && )` ❌

底层逻辑：

这是因为编译器在词法分析（`Lexing`）或极基础的符号流校验阶段，设定了一条铁律：“二元操作符的下一个非空 `Token` 绝对不能是另一个二元操作符”。一旦连续出现两个二元操作符，编译器连基本的语法树都无法生成，直接抛出 `Undeclared Identifier` 或词法解析错误。